

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES DISTRIBUIDAS [20701]

NIVEL / PARALELO: 7MO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 29-04-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: FCCDD-LAB-TIC-201

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Comprobación

N. HORAS: 9

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Construye aplicaciones distribuidas en capas que cumplan con estándares de calidad de software (ISO/IEC 25010), integrando mecanismos de seguridad y observando la ética del Ingeniero de Software.

Tema de la práctica

Comprobación experimental de la Ley de Amdahl mediante implementación y medición de un pipeline de procesamiento de datos en Apache Spark (PySpark) con tres transformaciones, comparación con procesamiento secuencial en pandas, y análisis cuantitativo del speedup obtenido vs. el speedup teórico máximo.

3 OBJETIVO

Objetivo General:

Comprobar experimentalmente la Ley de Amdahl implementando y midiendo el rendimiento de un pipeline distribuido de datos con Apache Spark sobre un dataset real ($\geq 500,000$ registros), comparando con el procesamiento secuencial equivalente.

Objetivos Específicos:

1. Implementar las mismas 5 transformaciones con pandas (secuencial) y PySpark (distribuido).
2. Medir los tiempos de ejecución con precisión de microsegundos (5 repeticiones por transformación, mediana).
3. Calcular el speedup experimental y compararlo con el speedup teórico de Amdahl.
4. Determinar la fracción serial de cada transformación y el límite práctico de paralelización.

4 FUNDAMENTO TEÓRICO

Desarrollar en LaTeX el fundamento teórico COMPLETO de la Unidad 4. Las ecuaciones de Amdahl y Gustafson DEBEN estar en entornos `equation/align` numerados. Los gráficos deben generarse con `matplotlib` e incluirse con `\includegraphics`. ≥ 2 referencias IEEE por subtema.

4.1 Fundamentos de Paralelismo y Ley de Amdahl

Desarrollar: paralelismo de datos y de tareas, Ley de Amdahl (1967) con derivación matemática completa, speedup teórico máximo, eficiencia del paralelismo, Ley de Gustafson-Barsis como alternativa, representación gráfica del speedup vs. número de procesadores para $p=0.5, 0.75, 0.9, 0.95$.

Referencias bibliográficas mínimas [IEEE]:

- [1] G. M. Amdahl, 'Validity of the single processor approach to achieving large scale computing capabilities,' in Proc. AFIPS Spring Joint Comput. Conf., 1967, vol. 30, pp. 483-485. doi: 10.1145/1465482.1465560.
- [2] J. L. Gustafson, 'Reevaluating Amdahl's law,' Commun. ACM, vol. 31, no. 5, pp. 532-533, May 1988. doi: 10.1145/42411.42415.

4.2 Paradigma MapReduce y Apache Hadoop

Desarrollar: modelo MapReduce de Dean y Ghemawat (2004), descripción formal de funciones Map y Reduce, flujo completo de ejecución (split, map, shuffle, sort, reduce), arquitectura Hadoop (HDFS y YARN), ventajas sobre procesamiento tradicional, y limitaciones para procesamiento iterativo.

Referencias bibliográficas mínimas [IEEE]:

[3] J. Dean and S. Ghemawat, 'MapReduce: Simplified data processing on large clusters,' Commun. ACM, vol. 51, no. 1, pp. 107-113, Jan. 2008. doi: 10.1145/1327452.1327492.

[4] T. White, Hadoop: The Definitive Guide, 4th ed. Sebastopol, CA, USA: O'Reilly Media, 2015. ISBN: 978-1-4919-0163-2.

4.3 Apache Spark: Arquitectura y Modelo de Programación

Desarrollar: arquitectura Spark (driver, cluster manager, executors), modelo RDD y su lineage para tolerancia a fallos, DataFrames y Dataset API, lazy evaluation vs. acciones, Spark execution model (DAG, stages, tasks), comparación de rendimiento Spark vs. MapReduce.

Referencias bibliográficas mínimas [IEEE]:

[5] M. Zaharia et al., 'Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,' in Proc. USENIX NSDI, 2012, pp. 15-28.

[6] B. Chambers and M. Zaharia, Spark: The Definitive Guide. Sebastopol, CA, USA: O'Reilly Media, 2018. ISBN: 978-1-4920-3251-9.

4.4 Computación en la Nube: Modelos y Proveedores

Desarrollar: modelos de servicio cloud (IaaS, PaaS, SaaS, FaaS) con definición NIST, características esenciales del cloud computing, comparación AWS vs. Azure vs. GCP para procesamiento distribuido (EMR, HDInsight, Dataproc), arquitecturas Lambda y Kappa.

Referencias bibliográficas mínimas [IEEE]:

[7] P. Mell and T. Grance, 'The NIST definition of cloud computing,' NIST, Spec. Publ. 800-145, Sep. 2011. doi: 10.6028/NIST.SP.800-145.

[8] B. Burns, Designing Distributed Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2025, ch. 8-10.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- LaTeX con: listings, booktabs, graphicx, amsmath, siunitx (unidades de tiempo)
- Kaggle API (opcional): `pip install kaggle` para descarga de datasets
- pandas 2.x, matplotlib 3.x, numpy 1.x (estándar Python para medición)
- PySpark 3.5.0: `pip install pyspark`
- Instalación local: Python 3.10+, Apache Spark 3.5.0, Java 11+ (requisito de Spar
- Databricks Community Edition (<https://community.cloud.databricks.com>) — gratui
- Google Colab (<https://colab.research.google.com>) — gratuito, 12 GB RAM, GPU op

Equipos

- Computadora con ≥ 16 GB RAM

Insumos y reactivos

- Conexión a Internet con ancho de banda superior a 2MBps

6 PROCEDIMIENTOS

Paso 1 Preparación del Entorno y Dataset

- Acceder a Google Colab o Databricks Community Edition. En Colab: `!pip install pyspark==3.5.0`
- Descargar dataset real $\geq 500,000$ registros relacionado con el dominio del PFC. Fuentes: Kaggle, INEC Ecuador (ecuadorencifras.gob.ec), datos.gob.ec, o generado con Faker (documentar fuente y licencia).
- Cargar en pandas y en PySpark DataFrame. Verificar número de registros y schema en ambos.

Paso 2 Implementación Secuencial con pandas

- Implementar 5 transformaciones con pandas (sin paralelismo): T1: filtrado por condición compuesta, T2: groupby + aggregation (≥ 3 funciones), T3: join de ≥ 2 DataFrames, T4: columna derivada compleja, T5: ordenamiento y top-N.
- Medir cada transformación con `time.perf_counter()`: 5 repeticiones, calcular mediana.
- Guardar resultados de cada transformación en CSV en `/data/pandas/`.

⚠ **Nota:** Ejecutar cada transformación de manera independiente para medir tiempos individuales correctamente.

Paso 3 Implementación Distribuida con PySpark

- Inicializar SparkSession: ``SparkSession.builder.config('spark.executor.instances', '4').getOrCreate()``
- Implementar las mismas 5 transformaciones con PySpark DataFrames. Resultados deben ser idénticos a pandas.
- Medir tiempos de la misma manera (5 ejecuciones, mediana).
- Para T3 (join): medir con 1, 2 y 4 executors para el análisis de speedup.
- Verificar resultados idénticos entre pandas y PySpark para cada transformación.

Paso 4 Análisis Cuantitativo — Ley de Amdahl

- Para cada transformación: $\text{Speedup}(S) = T_{\text{pandas}} / T_{\text{spark}}$.
- Para T3 con 1,2,4 executors: calcular fracción serial p con la fórmula inversa: $p = (1/S - 1/N) / (1 - 1/N)$.
- Calcular speedup teórico máximo predicho por Amdahl para $N \rightarrow \infty$ con el p obtenido.
- Generar con matplotlib: (a) barras: tiempos pandas vs. PySpark por transformación, (b) curva speedup vs. N con curva Amdahl superpuesta, (c) eficiencia $E = S/N$ vs. N . Exportar como PNG 300 DPI.

⚠ **Nota:** La comparación curva teórica vs. speedup real es el núcleo del análisis — debe documentarse con ecuaciones LaTeX numeradas.

Paso 5 Documento LaTeX

- Incluir: fundamento teórico U4 completo con ecuaciones Amdahl/Gustafson en entornos equation, código PySpark en `lstlisting`, tabla booktabs de tiempos medidos, tres figuras matplotlib con captions, análisis de cada figura, cálculo de speedup con trabajo matemático explícito, respuestas a las preguntas del Anexo A, conclusiones individuales, bibliografía (≥ 8 referencias IEEE).
- Mínimo 14 páginas de contenido. Compilar sin errores.

7 BIBLIOGRAFÍA

- P. Mell and T. Grance, 'The NIST definition of cloud computing,' NIST Spec. Publ. 800-145, 2011. doi: 10.6028/NIST.SP.800-145.
- J. L. Gustafson, 'Reevaluating Amdahl's law,' Commun. ACM, vol. 31, no. 5, pp. 532-533, May 1988. doi: 10.1145/42411.42415.
- B. Chambers and M. Zaharia, Spark: The Definitive Guide. Sebastopol, CA, USA: O'Reilly Media, 2018. ISBN: 978-1-4920-3251-9.
- M. Zaharia et al., 'Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,' in Proc. USENIX NSDI, 2012, pp. 15-28.
- J. Dean and S. Ghemawat, 'MapReduce: Simplified data processing on large clusters,' Commun. ACM, vol. 51, no. 1, pp. 107-113, Jan. 2008. doi: 10.1145/1327452.1327492.
- G. M. Amdahl, 'Validity of the single processor approach,' in Proc. AFIPS Spring Joint Comput. Conf., 1967, pp. 483-485. doi: 10.1145/1465482.1465560.

8 ANEXOS

- Calcula el speedup teórico máximo ($N \rightarrow \infty$) para la transformación T3 con el valor de p que obtuviste. ¿Con cuántos procesadores se alcanza el 90% del speedup máximo? Muestra el trabajo matemático completo.
- ¿Por qué algunas transformaciones muestran speedup < 1 (PySpark más lento que pandas)? ¿Cuál es el costo de overhead de Spark y qué tamaño mínimo de dataset justifica el uso de Spark?

- ¿Cómo implementarías el pipeline del PFC con Spark Structured Streaming si los datos llegaran en tiempo real? Describe la arquitectura.
- ¿Por qué Spark es más eficiente que MapReduce para procesamiento iterativo? Justifica con los conceptos de RDD lineage y lazy evaluation.

9 RESULTADOS DE ESTUDIANTES

10 RESPONSABLES

GUERRERO ULLOA GLEISTON CICERON
CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
CI: 1205316456
COORDINADOR DE CARRERA